

AIPSA-GM: Adaptive Islanded Parallel Simulated Annealing with Guided Migration

Pengxiang Li, Weilun Qiu, Jinkuo Ha

1. Introduction

Simulated Annealing (SA) is a widely used stochastic optimization method, but its inherently sequential nature limits its ability to leverage modern parallel architectures and slows convergence on multi-core systems.

Parallel island models provide a natural approach to parallelizing SA by running multiple independent instances (“islands”) that periodically exchange solutions. However, parallelizing SA is non-trivial due to the need to balance information sharing, communication overhead, and parallel scalability. Frequent migration may accelerate convergence but introduces synchronization and communication costs, potentially reducing parallel efficiency, while insufficient communication may lead to poor solution quality due to loss of diversity.

This project proposes **AIPSA-GM (Adaptive Islanded Parallel Simulated Annealing with Guided Migration)**, which integrates several features. First of all, we plan to implement **guided migration** in order to combine solution quality and diversity. Besides, we will also adopt **adaptive temperature control** in the algorithm based on the acceptance rate. To reduce synchronization overhead, we will use **asynchronous buffered migration**. We will also explore a bit on **topology-aware communication** (ring, fully connected, random k-neighbor) in later stage of the project.

In addition, we explore a **GPU-based extension** to contrast CPU-based coarse-grained parallelism (island model with communication) and GPU-based fine-grained parallelism (massively parallel evaluation), enabling a direct comparison of how hardware architecture impacts performance.

To maintain focus and depth, we evaluate two representative benchmarks. Firstly, we test and compare different methods in **Traveling Salesman Problem (TSP)** problem, which highlights migration and diversity effects. In addition, we apply them in **Rastrigin Function**, which highlights adaptive temperature behavior.

We also test the following hypotheses:

1. Guided migration (quality + diversity) outperforms random or best-only migration.
2. Adaptive temperature improves robustness compared to fixed cooling.
3. Asynchronous migration improves wall-clock performance by reducing synchronization overhead.
4. Communication topology affects convergence speed and parallel scalability.
5. GPU-based parallelism improves throughput but introduces different tradeoffs in solution quality.

2. Description & Implementation

2.1 Unified Problem Interface

To support modular experimentation across different benchmarks, we define a unified problem interface:

- `init_solution()`
- `neighbor(solution)`
- `cost(solution)`
- `distance(sol_a, sol_b)`

This abstraction decouples optimization logic from problem-specific implementations and enables consistent evaluation of migration strategies. For example, TSP uses 2-opt neighborhood and edge-based distance, while Rastrigin uses Gaussian perturbation and Euclidean distance.

2.2 Serial Baseline

We implement a serial SA baseline to establish correctness and provide a reference for performance comparison. The baseline includes:

- Initial temperature T_0
- Cooling schedule $T \leftarrow \alpha T$
- Inner loop length L
- Acceptance rule $\exp(-\Delta f/T)$
- Stopping criterion

We log iteration count, wall-clock time, current cost, best cost, and acceptance rate. This baseline serves as a foundation for analyzing speedup and parallel efficiency.

2.3 Parallel Versions

We implement three parallel variants using Python multiprocessing, representing different levels of parallelism and communication:

Baseline A: Independent Replicas Multiple SA processes run independently with no communication. This approach provides maximum parallelism but may suffer from poor solution quality due to lack of information sharing and reduced exploration diversity.

Baseline B: Static Synchronous Islands Islands execute for M steps and synchronize via a global barrier before exchanging solutions. While migration improves convergence, synchronization introduces idle time and limits scalability.

Baseline C: AIPSA-GM Each island maintains independent temperature and acceptance statistics. Adaptive temperature control is defined as:

$$T \leftarrow \begin{cases} 1.1T & r < r_{low} \\ 0.9T & r > r_{high} \\ \alpha T & \text{otherwise} \end{cases}$$

Guided migration is based on a utility function:

$$U(x, i) = \alpha q(x) + \beta d(x, i),$$

where $q(x)$ represents normalized solution quality and $d(x, i)$ represents normalized diversity relative to the recipient island i :

$$q(x) = \frac{(f_{\max} - f(x))}{f_{\max} - f_{\min} + \epsilon}$$

$$d(x, i) = \frac{\text{distance}(x, \text{current}_i)}{d_{\max}}$$

This formulation balances exploitation (high-quality solutions) and exploration (diversity), mitigating premature convergence.

2.3.1 Parallel Design Considerations

We analyze key parallel system challenges in our design:

Data Dependencies: Islands evolve independently, with dependencies introduced only during migration. This enables task-level parallelism with limited synchronization.

Load Balancing: Islands may progress at different rates due to stochastic behavior, leading to imbalance in convergence and computational workload.

Communication Cost: Migration introduces overhead that depends on frequency, topology, and number of islands. Excessive communication may reduce parallel efficiency and limit scalability as the number of islands increases.

Synchronization: Synchronous islands incur barrier overhead and idle time, while asynchronous designs remove global barriers at the cost of weaker consistency across islands.

2.4 Asynchronous Migration and Topology

To reduce synchronization overhead, we implement buffered asynchronous migration:

- Islands periodically push best solutions to a shared non-blocking queue.
- Islands non-blockingly check for incoming migrants.
- No global barrier is used.

This design reduces idle time caused by synchronization barriers and enables overlap between computation and communication, improving scalability.

We evaluate migration under different communication topologies, including ring, fully connected, and random k-neighbor, to study their impact on convergence speed and parallel efficiency.

2.5 GPU Extension

In addition to CPU-based multiprocessing, we implement a GPU-based parallel variant that exploits fine-grained data parallelism.

Multiple candidate solutions or independent SA trajectories are evaluated simultaneously on the GPU, enabling massively parallel cost computation without inter-process communication. This design offloads the dominant cost evaluation to the GPU, making it the primary computational component in large-scale experiments.

This contrasts with CPU-based island parallelism, which relies on coarse-grained task decomposition and communication between processes. The GPU approach offers higher throughput but limits information sharing, allowing us to study tradeoffs between parallel efficiency, scalability, and solution quality.

3. Evaluation

Since SA is stochastic, all experiments are repeated across multiple random seeds and reported using mean, variance, and success probability.

We evaluate performance in terms of both optimization quality and parallel efficiency, and analyze scalability using speedup, parallel efficiency, and Amdahl’s Law.

Experiment 1: Is Parallel SA Beneficial? Compare serial SA, independent replicas, synchronous islands, and AIPSA-GM. Metrics: best objective vs wall-clock time, time-to-target, speedup, and parallel efficiency. This experiment evaluates whether parallelization improves convergence and identifies scalability limits.

Experiment 2: Migration Policy Effect Compare random, best-only, quality-only, and quality+diversity migration. Metrics: convergence speed and inter-island diversity. This experiment studies the tradeoff between exploitation and exploration.

Experiment 3: Adaptive Temperature Compare fixed cooling vs adaptive cooling. Metrics: success rate, variance across seeds, and anytime performance. This experiment evaluates robustness and stability under stochastic behavior.

Experiment 4: Async vs Sync Compare synchronous and asynchronous migration. Metrics: wall-clock time, communication overhead, and CPU utilization. This experiment analyzes the impact of synchronization cost and evaluates whether asynchronous execution improves scalability by reducing idle time.

Experiment 5: Topology and Scalability Vary island count (1,2,4,8,16) and topology. Metrics: parallel efficiency, convergence rate, and diversity trends. This experiment evaluates scalability and examines how communication structure affects performance.

Experiment 6: CPU vs GPU Parallel SA Compare CPU-based island parallelism and GPU-based data-parallel SA. Metrics: throughput, time-to-target, parallel efficiency, and solution quality. This experiment studies tradeoffs between coarse-grained and fine-grained parallelism, and analyzes how hardware architecture affects scalability and optimization performance.